POLITECNICO
MILANO 1863

POLITECNICO DI MILANO

Scuola di Ingegneria Industriale e dell'Informazione

Prova Finale

Reti Logiche

Studente: Ambrogio Gao
Matricola: 214428
Corso di Laurea: Ingegneria Informatica
Anno Accademico: 2024–2025

Indice

1 Introduzione **2**
1.1 Descrizione sintetica 2
1.2 Specifiche 2
1.3 Interfaccia 3
1.4 Descrizione dei segnali 4

2 Architettura **6**
2.1 Segnali interni 6
2.2 Descrizione dei processi 6
2.3 Gli stati di FSM 7

3 Risultati sperimentali **13**
3.1 Sintesi 13
3.1.1 Report di Utilization 13
3.1.2 Report di timing 13
3.2 Simulazioni 14
3.2.1 Caso 1 – Reset e riavvio 14
3.2.2 Caso 2 – Fine ciclo e nuovo avvio 15
3.2.3 Caso 3 – Filtro d'ordine 3 con padding a zero 15
3.2.4 Caso 4 – Filtro d'ordine 5 con padding a zero 16

4 Conclusioni **18**

1 Introduzione

Nel progetto di Reti Logiche dell'anno accademico 2024/2025 ho sviluppato un modulo hardware in linguaggio VHDL capace di interfacciarsi con una memoria esterna. L'obiettivo era rispettare le specifiche fornite nel bando e verificare, tramite simulazioni e sintesi, il corretto funzionamento del sistema. Il lavoro si è concentrato soprattutto sulla progettazione della macchina a stati finiti (FSM) e sulla gestione ordinata delle fasi di lettura, elaborazione e scrittura dei dati in memoria.

1.1 Descrizione sintetica

Il sistema legge da memoria un messaggio composto da K parole W (valori tra -128 e +127, in complemento a 2), applica un filtro differenziale e salva in memoria la sequenza risultante di K parole R. I valori di R che superano i limiti dell'intervallo [-128, +127] vengono saturati al valore massimo o minimo consentito (es. se R = -196, diventa -128).

1.2 Specifiche

Tutti i dati da utilizzare sono memorizzati sequenzialmente a partire da un indirizzo specificato (`ADD`). Il sistema legge inizialmente 17 byte $(2 + 1 + 14)$ che contengono le informazioni necessarie alla configurazione del filtraggio: la lunghezza della sequenza da elaborare, la selezione del filtro differenziale e i coefficienti dei filtri stessi.

- **K1 e K2**: due byte che rappresentano rispettivamente il byte più significativo e il meno significativo della lunghezza K della sequenza di dati W. Il valore minimo di K è 7. Si assume che i dati forniti nel Test Bench garantiscano sempre la validità degli indirizzi di memoria coinvolti.
- **S**: un byte che indica quale filtro differenziale applicare. La selezione è vincolata al bit meno significativo:
  - se $S[0] = 0$, viene utilizzato il filtro di ordine 3;
  - se $S[0] = 1$, viene utilizzato il filtro di ordine 5.
- **C1–C14**: quattordici byte che contengono i coefficienti dei due filtri. In particolare, si hanno i seguenti valori:

$$\textbf{Filtro 3:} \quad c = [0,\ -1,\ 8,\ 0,\ -8,\ 1,\ 0], \quad n = 12$$

$$\textbf{Filtro 5:} \quad c = [1,\ -9,\ 45,\ 0,\ -45,\ 9,\ -1], \quad n = 60$$

I coefficienti possono variare a ogni nuova elaborazione, poiché vengono letti direttamente dalla memoria.

Dopo i 17 byte iniziali, vengono letti K byte corrispondenti alle parole W_i da elaborare, con valori compresi tra -128 e $+127$ in rappresentazione a complemento a 2. A ciascun valore W_i viene applicato il filtro selezionato, producendo una sequenza filtrata R_i, che viene riscritta in memoria a partire dall'indirizzo:

$$\texttt{ADD} + 17 + K$$

Il filtraggio è definito dalla seguente formula generale:

$$f'(i) = \frac{1}{n} \sum_{j=-l}^{+l} C_j \cdot f[i+j]$$

dove $l = 2$ per il filtro di ordine 3 e $l = 3$ per il filtro di ordine 5. I valori al di fuori della sequenza ($i - l < 0$ o $i + l > K$) vengono considerati pari a zero.

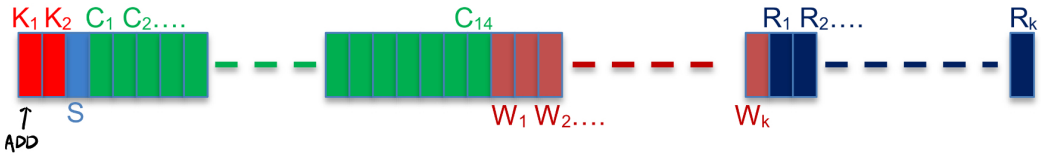

Figura 1: Schema del filtraggio con K_1, K_2, i coefficienti C_j e i dati W_i.

Approssimazione della divisione La divisione per il fattore di normalizzazione n viene approssimata mediante somme di shift a destra:

$$\frac{1}{12} \approx \frac{1}{16} + \frac{1}{64} + \frac{1}{256} + \frac{1}{1024}, \quad \frac{1}{60} \approx \frac{1}{64} + \frac{1}{1024}$$

Ogni shift a destra equivale a una divisione per due e produce troncamenti verso il basso per numeri negativi e verso zero per numeri positivi. Per ridurre l'errore di arrotondamento, si aggiunge $+1$ dopo ciascuno shift se il valore è negativo.

1.3 Interfaccia

L'interfaccia del componente è la seguente:

```
entity project_reti_logiche is
    port (
        i_clk      : in  std_logic;
        i_rst      : in  std_logic;
        i_start    : in  std_logic;
        i_add      : in  std_logic_vector(15 downto 0);

        o_done     : out std_logic;

        o_mem_addr : out std_logic_vector(15 downto 0);
        i_mem_data : in  std_logic_vector(7 downto 0);
        o_mem_data : out std_logic_vector(7 downto 0);
        o_mem_we   : out std_logic;
        o_mem_en   : out std_logic
    );
end project_reti_logiche;
```

1.4 Descrizione dei segnali

- **i_clk**: segnale di *clock* in ingresso, fornito dal Test Bench, utilizzato per sincronizzare tutte le operazioni del modulo.
- **i_rst**: segnale di *reset* attivo alto, che inizializza la macchina a stati rendendola pronta a ricevere il primo impulso di **i_start**.
- **i_start**: segnale di avvio dell'elaborazione, generato dal Test Bench. Rimane attivo per tutta la durata dell'esecuzione e viene riportato basso solo al termine, quando `o_done` diventa alto.
- **i_add**: vettore di ingresso (16 bit) che rappresenta l'indirizzo di memoria di partenza, corrispondente al primo byte dei 17 byte iniziali (preambolo) che precedono la sequenza di dati numerici da processare.
- **o_done**: segnale di uscita che indica la conclusione dell'elaborazione dei dati e la disponibilità dei risultati in memoria.
- **o_mem_addr**: vettore di uscita (16 bit) che fornisce l'indirizzo della cella di memoria con cui il modulo intende comunicare.
- **i_mem_data**: vettore di ingresso (8 bit) che trasporta il dato proveniente dalla memoria a seguito di una richiesta di lettura.

- **o_mem_data**: vettore di uscita (8 bit) che contiene il dato da scrivere in memoria in seguito all'elaborazione.
- **o_mem_we**: segnale di controllo *Write Enable*. Quando è impostato a '1' abilita la scrittura in memoria; per la lettura si tiene '0' e si usa `o_mem_en=1` per abilitare la comunicazione.
- **o_mem_en**: segnale di *Enable* generale, necessario per attivare la comunicazione con la memoria, sia in fase di lettura sia in fase di scrittura.

2 Architettura

L'architettura del modulo si basa su una macchina a stati finiti (FSM) che coordina in modo ordinato le fasi di lettura, elaborazione e scrittura dei dati. Il sistema lavora in modo completamente sincrono, controllato dal segnale di clock, e segue una sequenza di stati che assicura il corretto avanzamento dell'elaborazione.

2.1 Segnali interni

Per la gestione delle operazioni sono stati introdotti diversi segnali interni, utili sia per la logica di controllo sia per il percorso dati:

```
signal curr_state , next_state : state_type;
signal K : std_logic_vector(15 downto 0);
signal S : std_logic;
signal mod3 : std_logic_vector(39 downto 0);
signal mod5 : std_logic_vector(55 downto 0);
signal sequenceNumb_mod3 : std_logic_vector(39 downto 0);
signal sequenceNumb_mod5 : std_logic_vector(55 downto 0);
signal counter : unsigned(15 downto 0);
signal reg_load : std_logic;
```

- `curr_state`, `next_state`: stato corrente e stato successivo della FSM;
- `K` (`std_logic_vector(15 downto 0)`): lunghezza della sequenza da elaborare;
- `S` (`std_logic`): selezione del filtro differenziale (0 → ordine 3, 1 → ordine 5);
- `mod3`, `mod5`: registri che memorizzano i coefficienti dei filtri;
- `sequenceNumb_mod3`, `sequenceNumb_mod5`: finestre di campioni utilizzate durante l'elaborazione;
- `counter` (`unsigned(15 downto 0)`): contatore dell'indice di lettura/scrittura;
- `reg_load`: segnale che abilita il caricamento dei registri interni.

2.2 Descrizione dei processi

L'architettura è organizzata in più processi, ciascuno con un compito ben definito.

- **Processo 1 – Sincronizzazione degli stati.** Sensibile a `i_clk` e `i_rst`, aggiorna lo stato corrente della FSM. Quando `i_rst` è alto, il sistema torna nello stato iniziale `S0`; ad ogni fronte di salita del clock, `curr_state` viene aggiornato con `next_state`.

- **Processo 2 – Transizioni di stato.** È un processo combinatorio che determina il prossimo stato in base ai segnali `i_start`, `counter` e `S`. La macchina passa progressivamente dagli stati di inizializzazione (`S0`, `S1`) a quelli di elaborazione e scrittura (`S2`–`S5`), fino a concludere in `S6`, che segnala la fine dell'esecuzione.

- **Processo 3 – Generazione dei segnali di controllo.** In base allo stato corrente vengono definiti i segnali di controllo per la memoria: in `S1` si abilitano lettura e caricamento, in `S3` la scrittura, in `S4` l'avanzamento dell'indirizzo, mentre in `S6` viene portato alto `o_done`.

- **Processo 4 – Calcolo dell'indirizzo di memoria.** Calcola dinamicamente l'indirizzo di memoria `o_mem_addr`. Durante la scrittura (`S3`) l'indirizzo è:

  $$o_mem_addr = i_add + counter + K - \Delta,$$

  dove $\Delta = 3$ per il filtro di ordine 3 e $\Delta = 4$ per quello di ordine 5. Negli altri stati, l'indirizzo coincide con `i_add + counter`.

- **Processo 5 – Registrazione dei parametri e dei dati.** È un processo sincrono che memorizza i parametri letti dalla memoria e aggiorna il contatore. Durante il reset o quando `i_start = 0`, tutti i registri vengono azzerati. Quando `reg_load` è attivo, vengono caricati in sequenza i due byte di `K`, il bit `S`, i coefficienti dei filtri e i valori della sequenza `W`.

- **Processo 6 – Calcolo del risultato.** Esegue il filtraggio combinatorio moltiplicando i campioni della finestra per i coefficienti del filtro selezionato. Per l'ordine 3 vengono eseguiti cinque prodotti e la normalizzazione tramite shift a destra (4, 6, 8 e 10 bit); per l'ordine 5 sette prodotti e due shift (6 e 10 bit). Se la somma è negativa viene applicata una correzione per compensare il troncamento e il risultato è saturato nell'intervallo $[-128, +127]$.

2.3 Gli stati di FSM

La macchina a stati finiti coordina le fasi di lettura, calcolo e scrittura del filtro differenziale. Di seguito si riportano, per ciascuno stato, le condizioni di ingresso/uscita e le azioni eseguite.

- **S0 — Attesa avvio**

– *Ingresso*: stato di riposo dopo `reset` oppure al termine di una elaborazione.

– *Azioni*: nessuna azione; tutti i segnali di controllo a `'0'`.

– *Uscita*: se `i_start='1'` allora passa a **S1**; altrimenti rimane in **S0**.

- **S1 — Lettura parametri e prime finestre**

  – *Azioni*:

    * abilita la memoria e il caricamento: `o_mem_en='1'`, `reg_load='1'`;

    * tramite il processo sincrono di caricamento: acquisisce i 2 byte di `K`, il bit `S`, i coefficienti del filtro selezionato (`mod3` o `mod5`) e inizia a riempire la finestra dei campioni (`sequenceNumb_mod3` o `sequenceNumb_mod5`);

    * lo `skip` dei coefficienti non necessari è gestito dal contatore: se `S='1'` salta direttamente all'area `mod5` (`counter:=10`), se `S='0'` salta l'area `mod5` (`counter:=17`).

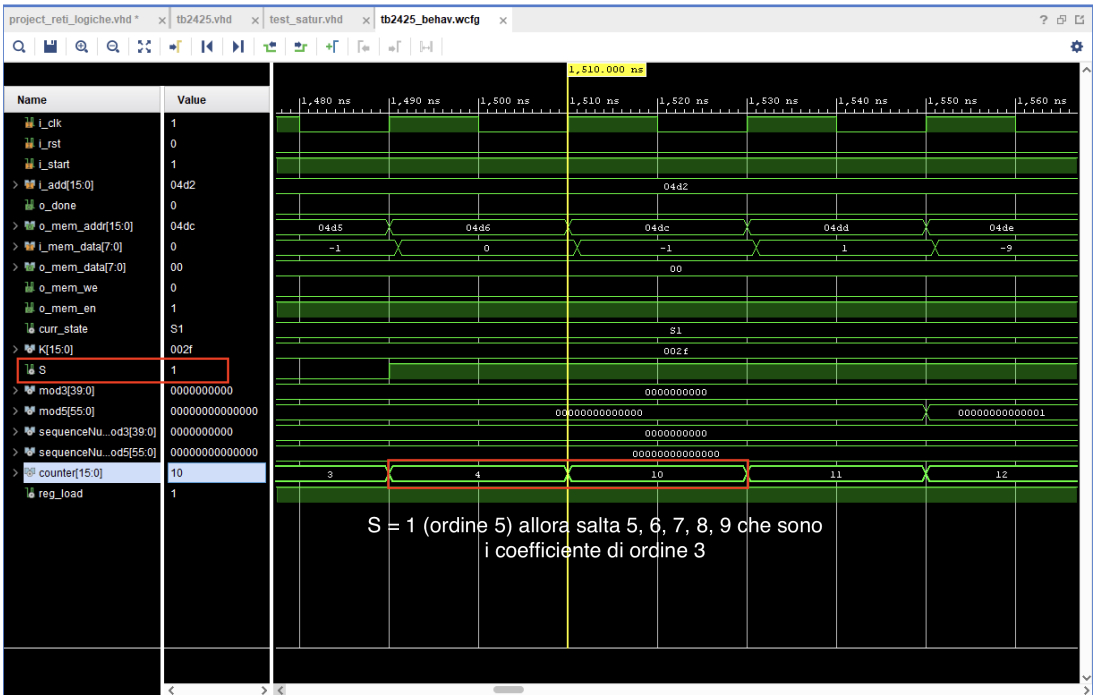

Figura 2: Esempio salta indirizzo.

  – *Transizione*:

    * per filtro ordine 3 (`S='0'`): appena la finestra iniziale contiene i primi *3* campioni utili (condizione `to_integer(counter)=20`) ⇒ **S2**;

* per filtro ordine 5 (`S='1'`): appena la finestra iniziale contiene i primi *4* campioni utili (condizione `to_integer(counter)=21`) ⇒ **S2**;

Figura 3: Ordine 5: caricamento al quarto campione

* altrimenti rimane in **S1**.

- **S2 — Attesa di stabilizzazione**
  - *Azioni*: nessun segnale attivo; serve un ciclo di attesa per permettere alla logica combinatoria del calcolo di stabilizzarsi.
  - *Uscita*: passa sempre a **S3**.
- **S3 — Scrittura del risultato**
  - *Azioni*: abilita la scrittura in RAM: `o_mem_en='1'`, `o_mem_we='1'`. L'indirizzo di scrittura è calcolato come

$$\texttt{o_mem_addr} = \texttt{i_add} + \texttt{counter} + K - \Delta,$$

    con $\Delta = 3$ per il filtro ordine 3 e $\Delta = 4$ per il filtro ordine 5, così da depositare i risultati `R` subito dopo la sequenza `W`. Il dato scritto è `o_mem_data`, ottenuto dal processo combinatorio (moltiplica–somma, normalizzazione per shift, correzione dei negativi, saturazione a 8 bit).
  - *Transizione*:
    * se è stato scritto l'ultimo risultato (`counter = K + 20` per ordine 3, `counter = K + 21` per ordine 5) ⇒ **S6**;

* altrimenti, se si è in coda alla sequenza (`counter` $\geq$ `K + 18`) $\Rightarrow$ **S5** (caricamento degli zeri di coda);

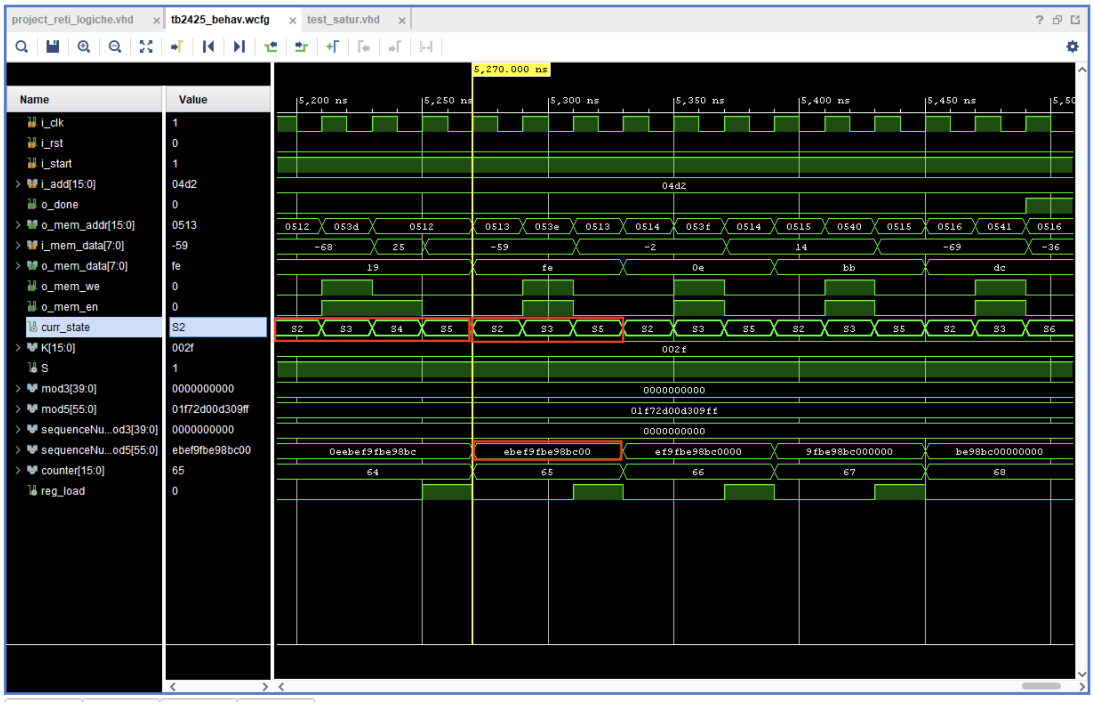

Figura 4: Carica degli zeri e salta lo stato di **S4**.

* altrimenti $\Rightarrow$ **S4** (avanzamento lettura).

- **S4 — Avanzamento indirizzo di lettura**
  - *Azioni*: abilita la memoria per preparare la lettura del prossimo byte `W`: `o_mem_en='1'`. L'indirizzo di lettura è `i_add + counter`.
  - *Uscita*: passa a **S5**.
- **S5 — Caricamento del nuovo campione nella finestra**
  - *Azioni*: `reg_load='1'` per spostare la finestra (`sequenceNumb_mod3 <= sequenceNumb_mod3(31 downto 0) & i_mem_data`). Quando si supera la fine della sequenza, il byte inserito è `"00000000"`. Il contatore si incrementa automaticamente, con la salvaguardia di non saltare il campione nella transizione **S1**$\rightarrow$**S2**.

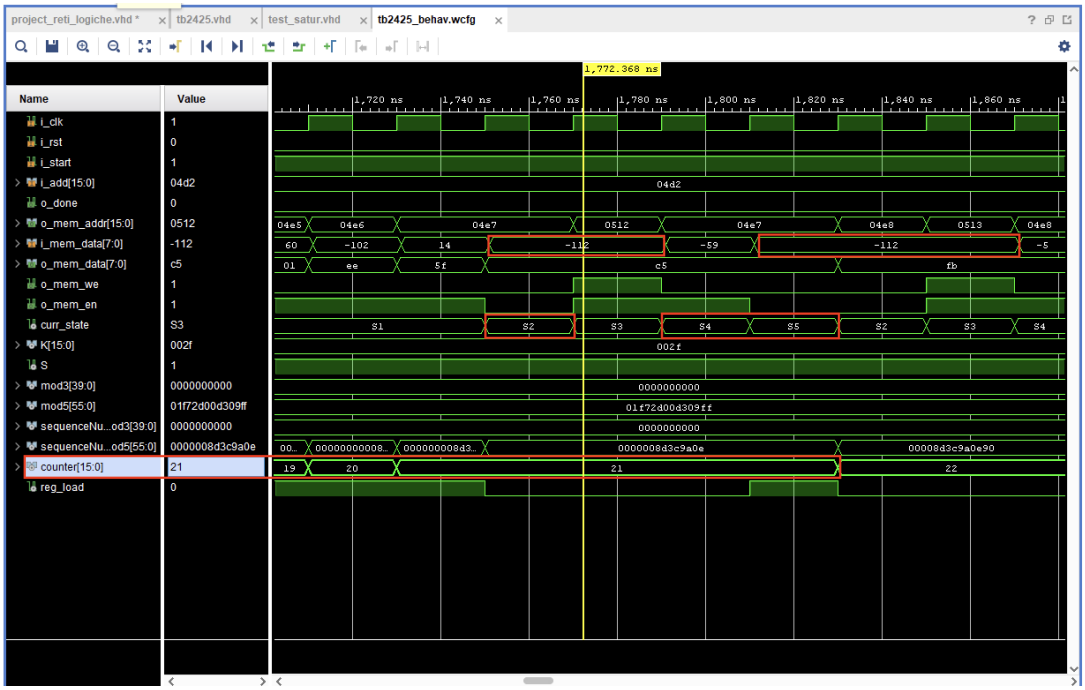

Figura 5: Esempio di non saltare numero 21 (ordine 5).

  - *Uscita*: torna a **S2** per il successivo ciclo *calcolo* → *scrittura*.

- **S6 — Fine elaborazione**
  - *Azioni*: alza il segnale di fine: `o_done='1'`.
  - *Uscita*: resta in **S6** finché `i_start` non torna a `'0'`; quindi rientra in **S0**.

- **Diagramma di FSM**

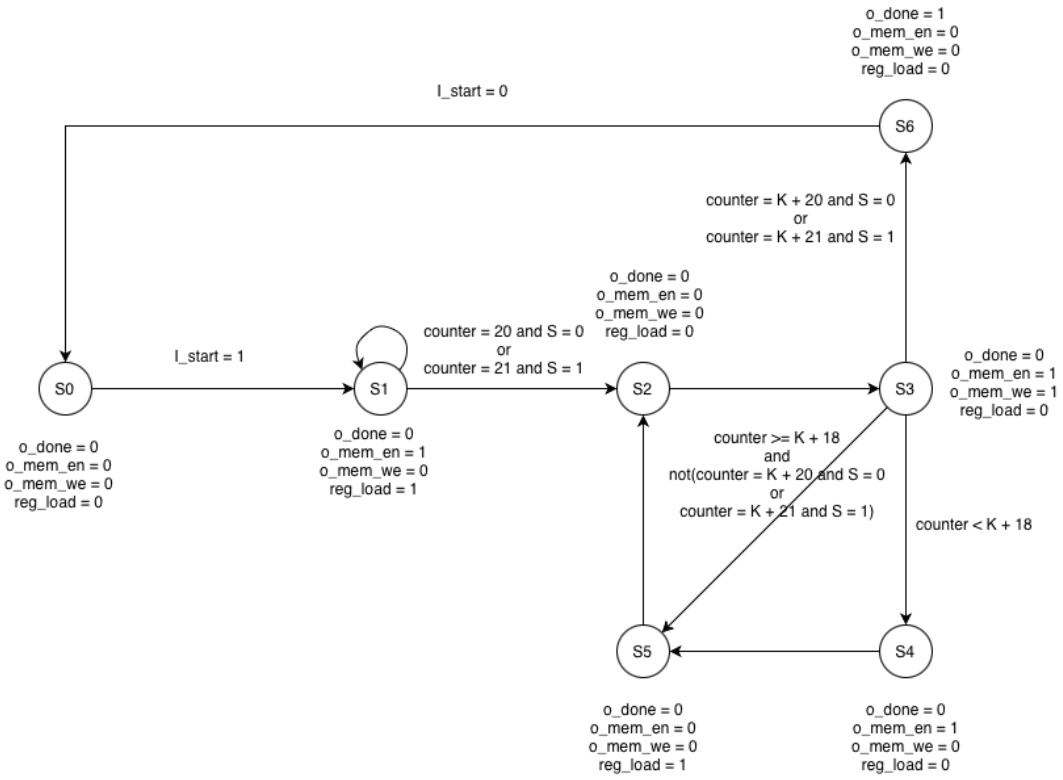

Figura 6: Macchina a stati finiti.

3 Risultati sperimentali

In questa sezione vengono riportati i principali risultati ottenuti dopo la sintesi e le simulazioni del modulo. L'obiettivo era verificare la correttezza funzionale e temporale del progetto e confrontare il comportamento simulato con le specifiche previste.

3.1 Sintesi

Il modulo è stato sintetizzato e simulato correttamente sia in pre-sintesi sia in post-sintesi, utilizzando *Vivado 2018.3.1* sul dispositivo `xc7a200t-fbg484-1`. I risultati confermano che l'architettura proposta è pienamente compatibile con i vincoli del progetto.

3.1.1 Report di Utilization

La sintesi mostra un utilizzo molto ridotto delle risorse logiche e, soprattutto, l'assenza di latch, segno che la macchina a stati è interamente di tipo sincrono. La tabella seguente riassume i dati principali:

Tabella 1: Risorse logiche principali (post-sintesi).

Metrica	Valore	Util%
Slice LUTs (totali)	1156	0.86 %
LUT as Logic	1156	0.86 %
LUT as Memory	0	0.00 %
Slice Registers	228	0.08 %
Reg. as Flip–Flop	228	0.08 %
Reg. as Latch	0	0.00 %
DSPs	0	0.00 %
Block RAM (BRAM)	0	0.00 %
Bonded IOB	54	18.95 %
BUFGCTRL	1	3.13 % (di 32)

3.1.2 Report di timing

Anche le prestazioni temporali risultano ampiamente soddisfatte. Il vincolo di periodo fissato a `20 ns` (clock = 50 MHz) è rispettato con un margine positivo di circa `+14 ns`. Il cammino peggiore presenta un *Data Path Delay* di 5.864 ns, suddiviso in 48.7 % di logica e 51.3 % di routing, per un totale di 8 livelli logici (`5 CARRY4 + 3 LUT6`). Questo conferma una buona distribuzione del carico e nessun percorso critico significativo.

Tabella 2: Timing principale (cammino peggiore).

Metrica	Valore
Vincolo di periodo	20.000 ns (50 MHz)
Slack (MET)	+**13.985 ns** (rispettato)
Data Path Delay	5.864 ns
Logica / Routing	2.855 ns (48.7%) / 3.009 ns (51.3%)
Livelli logici	8 (CARRY4=5, LUT6=3)
Sorgente / Destinazione	`K_reg[3]/C` → `curr_state_reg[0]/D`

3.2 Simulazioni

Le simulazioni hanno avuto lo scopo di verificare il comportamento funzionale del modulo: la gestione della finestra scorrevole, la normalizzazione tramite shift con correzione dei valori negativi, la saturazione entro i limiti $[-128, 127]$ e il corretto protocollo di lettura/-scrittura con la memoria. Tutti i test sono stati eseguiti sia in pre-sintesi (modellazione comportamentale) sia, dove utile, in post-sintesi con annotazione dei ritardi.

3.2.1 Caso 1 – Reset e riavvio

In questo test è stato forzato `i_rst = '1'` per verificare che la FSM si inizializzasse correttamente. La waveform mostra che, durante il reset, tutte le uscite vengono poste a zero e i registri interni sono azzerati. Dopo il rilascio del reset, con `i_start = '1'`, la macchina riparte dal primo stato (`S1`) e procede con la sequenza di lettura e scrittura prevista, confermando un corretto riavvio.

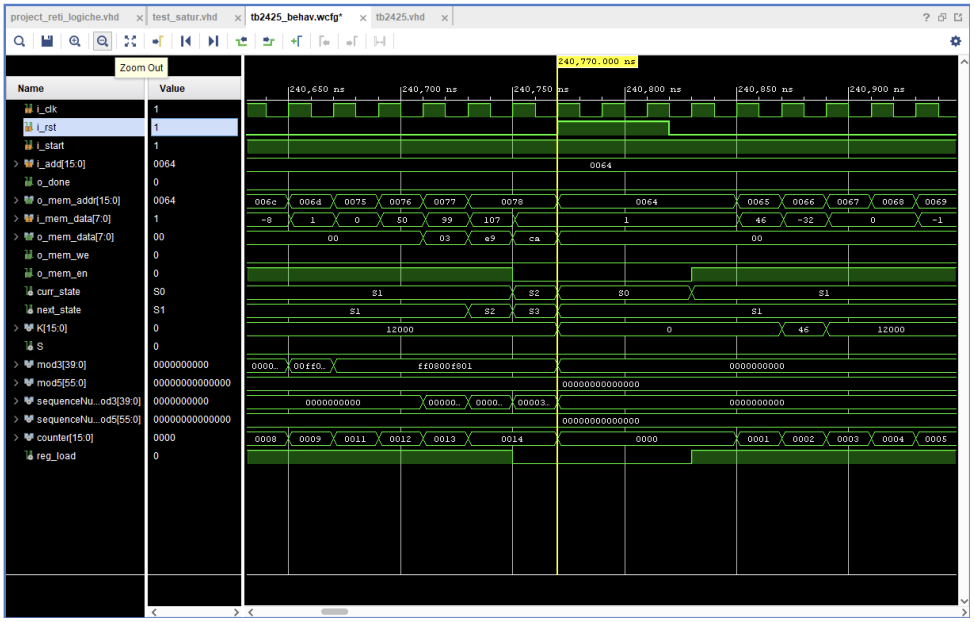

Figura 7: TB reset e riavvio.

3.2.2 Caso 2 – Fine ciclo e nuovo avvio

Questo scenario verifica che, al termine del primo ciclo di elaborazione, la FSM torni nello stato iniziale `S0` e possa avviare un nuovo periodo senza reset esterno. Il segnale `o_done` viene alzato nello stato `S6`, poi riportato basso insieme a `i_start`, consentendo un nuovo ciclo di elaborazione ordinato.

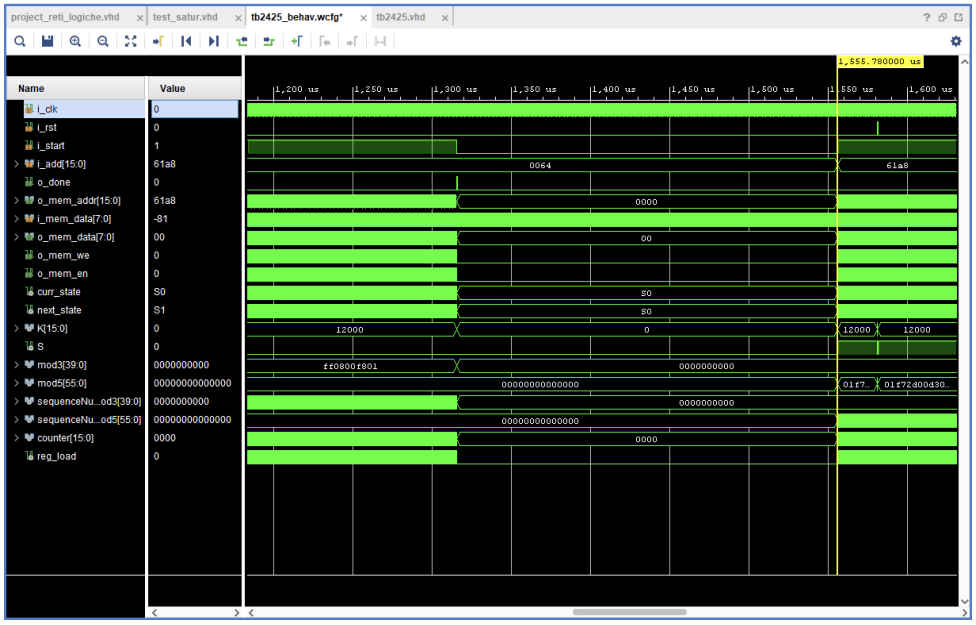

Figura 8: Chiusura del primo ciclo (`S6`, `o_done=1`) e riavvio del successivo tramite `i_start`.

3.2.3 Caso 3 – Filtro d'ordine 3 con padding a zero

Con `S = 0` e `K = 24`, la FSM applica il filtro di ordine 3 utilizzando una finestra di 5 campioni ($l = 2$). Ai bordi della sequenza vengono aggiunti zeri per completare la finestra. Nel tracciato si osserva la ripetizione regolare della sequenza `S4 → S5 → S2 → S3`, con scritture corrette dei risultati e termine in `S6` quando `counter = K + 20`.

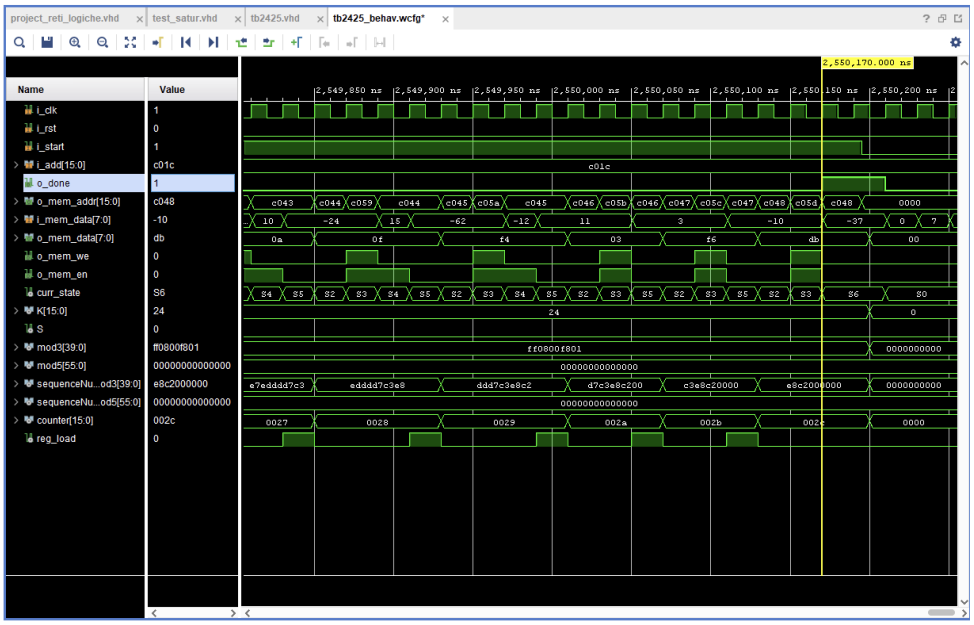

Figura 9: Filtro ordine 3 con coda a zero; al raggiungimento di `counter=K+20` la FSM entra in `S6` e alza `o_done`.

3.2.4 Caso 4 – Filtro d'ordine 5 con padding a zero

Con `S = 1`, il modulo utilizza la finestra di 7 campioni ($l = 3$). La waveform conferma il corretto avanzamento della FSM e la scrittura dei risultati filtrati. Si osservano inoltre valori saturati a $+127$ e -128, a conferma della corretta gestione della normalizzazione e del *clamp* a 8 bit.

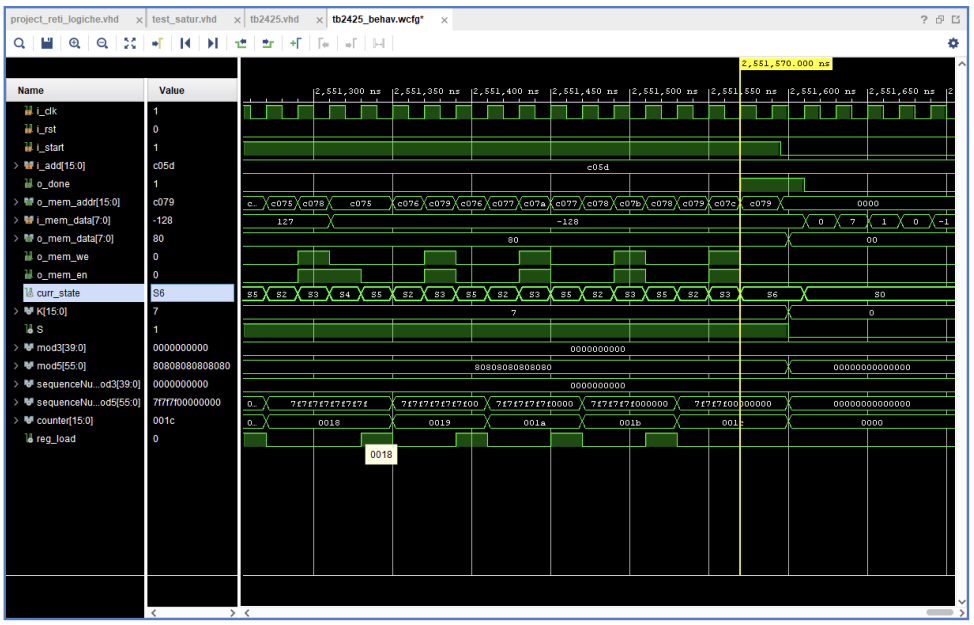

Figura 10: Filtro d'ordine 5 (`S=1`), padding a zero e termine in `S6` (`o_done`=1).

Caso 5 – Saturazione positiva

Per `K = 7` e filtro d'ordine 5, alcuni valori di uscita raggiungono il limite superiore (+127). Il comportamento è coerente con la formula di scrittura

$$o_mem_addr = i_add + counter + K - 4$$

e dimostra che la logica di saturazione funziona correttamente.

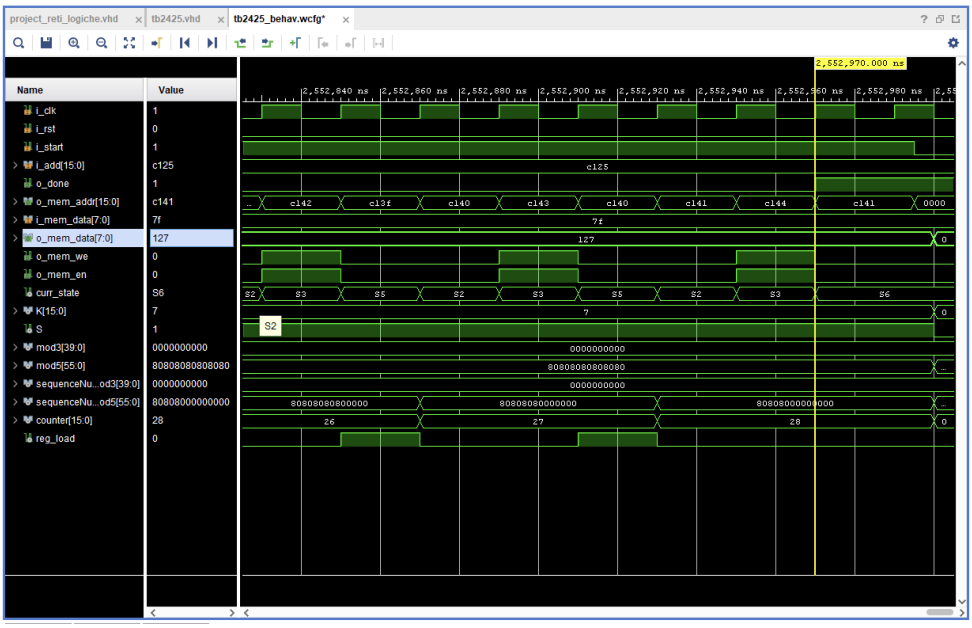

Figura 11: Filtro ordine 5 con saturazione a +127 e padding finale; terminazione in `S6`.

Caso 6 – Saturazione negativa

Con le stesse impostazioni del caso precedente, ma coefficienti di segno opposto, alcuni risultati raggiungono il limite inferiore (−128). Anche in questo caso la FSM completa il ciclo correttamente portando `o_done` a 1 e tornando infine allo stato `S0`.

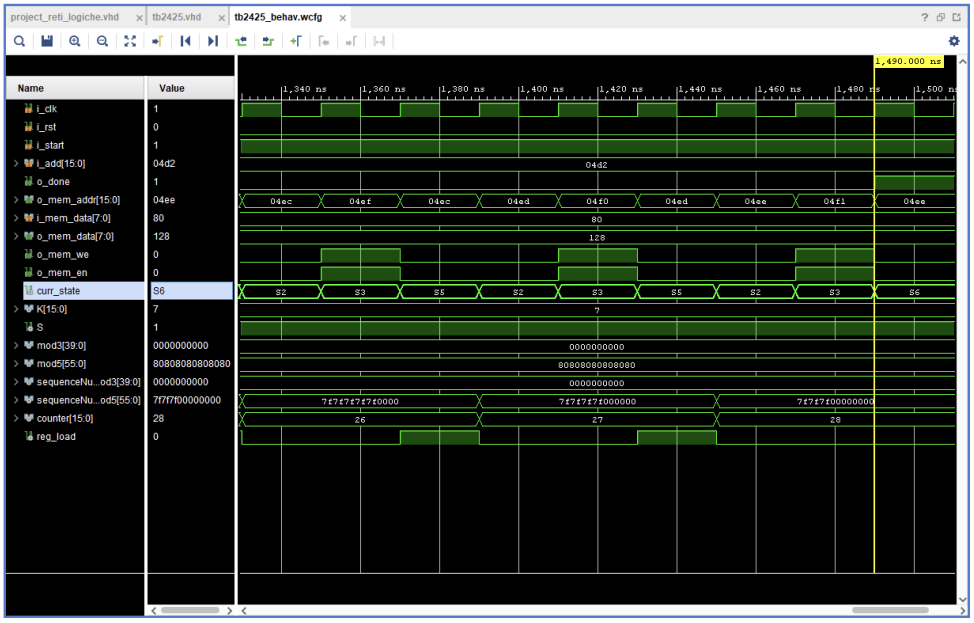

Figura 12: Filtro ordine 5 (`S=1`) con saturazione a -128 e termine in `S6`.

4 Conclusioni

In conclusione, il modulo `project_reti_logiche` realizza correttamente il filtraggio differenziale richiesto, applicando un filtro di ordine 3 o 5 e riscrivendo i risultati in memoria con saturazione nell'intervallo $[-128, +127]$.

La separazione tra logica di controllo e percorso dati, insieme all'uso di registri per la finestra scorrevole, ha permesso di ottenere un comportamento stabile e prevedibile. Dal punto di vista della sintesi, le risorse utilizzate risultano estremamente contenute (LUT e FF $< 1\%$) e il vincolo temporale di 20 ns è stato ampiamente rispettato.

Le simulazioni hanno confermato la correttezza del funzionamento in tutti gli scenari testati — dal reset iniziale alla saturazione dei valori — dimostrando la solidità complessiva del progetto.